

Semgrep CE로 Android 보안 점검

로컬 룰 2개로 후보를 검출하고, 오탐 분류를 시로 보강하는 운영 흐름을 정리합니다.

이번 발표의 범위

- 사용 범위: Semgrep Community Edition CLI + 직접 작성한 로컬 룰
- 제외 범위: Registry, 외부 rule pack, AppSec Platform/Pro, p/java, p/kotlin, p/default
- 이번 발표에서는 룰 2개 만 다룹니다.
- 각 묶음은 같은 순서로 설명합니다.

문제 설명

→ 문제 코드

→ 안전한 기본형 / 비교 코드

→ Semgrep 규칙 설명

→ 검출 결과

→ 오탐 사례

→ AI 분류 기준

→ AI 검토 입력

로컬 실행 구조

```
android-project/  
├── app/src/main/kotlin/com/example/PendingIntentLab.kt  
├── app/src/main/java/com/example/LegacyCrypto.java  
└── rules/android-local.yml
```

```
semgrep scan --metrics=off \  
--config rules/android-local.yml \  
app/src/main
```

- 두 룰은 한 파일 `rules/android-local.yml` 안에 묶어 두고, 새 룰이 생기면 같은 파일에 append 합니다.
- 목적은 "완성형 판정기"가 아니라, 후보를 빠르게 추출하고 분류 시간을 줄이는 데 있습니다.

예시 1: 무엇을 점검하나

android-pendingintent-flag-mutable

- `PendingIntent` 는 단순한 `Intent` 복사본이 아니라, 내 앱 권한으로 나중에 실행할 수 있는 시스템 토큰에 가깝습니다.
- 다른 앱이나 시스템 UI에 넘겨도, 실행 시에는 원래 `PendingIntent` 를 만든 앱 정체성으로 동작할 수 있습니다.
- 그래서 리뷰 포인트는 "이 Intent가 어디로 가는가"뿐 아니라, "누가 나중에 어떤 값을 채워 실행할 수 있는가"까지 보는 데 있습니다.

App A creates `PendingIntent`
-> System UI or App B stores token
-> Later `send()` / notification action
-> Action runs with App A identity

예시 1: 어떤 조합이 위험한가

패턴	왜 위험한가
<code>FLAG_MUTABLE</code> + empty / implicit <code>Intent</code>	action, data, component, extras가 받은 쪽에서 변형될 여지가 커집니다.
<code>requestCode = 0</code> 반복 + <code>FLAG_UPDATE_CURRENT</code>	기존 토큰과 extras가 충돌하거나 덮어써질 수 있습니다.
notification action이 바로 민감 작업 수행	잠금화면·외부 표면과 결합될 때 오용 여지가 생깁니다.

- 특히 `FLAG_MUTABLE` + implicit `Intent` 조합은 위험도가 높습니다.
- Android 14 / target SDK 34+에서는 이 조합이 기본적으로 막히는 방향으로 강화되었습니다.

예시 1: 코드 리뷰 빨간불 패턴

```
// 이 룰이 1차로 잡는 패턴 (FLAG_MUTABLE)
```

```
PendingIntent.getActivity(context, 0, Intent(), PendingIntent.FLAG_MUTABLE)
```

```
PendingIntent.getBroadcast(context, 0, Intent("SOME_ACTION"), PendingIntent.FLAG_MUTABLE)
```

```
PendingIntent.getService(context, 0, intent, PendingIntent.FLAG_MUTABLE)
```

```
// 룰이 직접 잡지는 않지만 함께 봐야 할 주변 신호
```

```
PendingIntent.getActivity(context, 0, intent, PendingIntent.FLAG_UPDATE_CURRENT)
```

```
PendingIntent.getBroadcast(context, 0, intent, PendingIntent.FLAG_CANCEL_CURRENT)
```

- 같이 보면 더 위험한 주변 신호 (룰이 직접 잡지는 않음):

- `FLAG_GRANT_READ_URI_PERMISSION` / `WRITE` 가 같이 붙어 URI 권한 흐름이 따라감
- 내부 `Receiver` / `Service` 가 extras를 그대로 신뢰
- 민감 작업인데 `FLAG_ONE_SHOT` 이 없어 재실행 가능

예시 1: 문제 코드

```
package com.example

class PendingIntentLab {
    fun buildDangerousDelete(context: Context, sensitiveFileUri: Uri): PendingIntent {
        val deleteIntent = Intent("com.example.ACTION_DELETE_FILE").apply {
            data = sensitiveFileUri
            addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
            putExtra("fileId", "report-4412")
        }

        return PendingIntent.getBroadcast(
            context,
            0,
            deleteIntent,
            PendingIntent.FLAG_MUTABLE or PendingIntent.FLAG_UPDATE_CURRENT
        )
    }
}
```

- 이 함수는 학습용으로 5가지 안티패턴이 한 줄에 겹친 합성 예시입니다.
- `FLAG_MUTABLE` : 받은 쪽이 미채워진 필드나 extras를 보강할 수 있습니다.
- `Intent("ACTION")` : explicit component가 아니라 implicit `Intent` 입니다.
- `requestCode = 0` : 다른 토큰과 충돌하기 쉬운 기본값입니다.

예시 1: 비교적 안전한 기본형

```
val notificationId = 1042
```

```
val intent = Intent(context, DeleteConfirmActivity::class.java).apply {  
    action = "com.example.ACTION_DELETE_FILE"  
    setPackage(context.packageName)  
    putExtra("fileId", fileId)  
}
```

```
val pi = PendingIntent.getActivity(  
    context,  
    notificationId,  
    intent,  
    PendingIntent.FLAG_IMMUTABLE or  
    PendingIntent.FLAG_UPDATE_CURRENT or  
    PendingIntent.FLAG_ONE_SHOT  
)
```

- 기본 안전선:
 - explicit component
 - `FLAG_IMMUTABLE`
 - unique `requestCode`
 - 최소 extras
 - 실제 삭제/승인은 대상 화면이나 receiver에서 다시 검증

예시 1: Semgrep 규칙

```
rules:  
- id: android-pendingintent-flag-mutable  
  languages: [java, kotlin]  
  severity: ERROR  
  message: >  
    Mutable PendingIntent입니다.  
    대부분은 FLAG_IMMUTABLE을 사용해야 하며, mutable이 꼭 필요하다  
    explicit component/package, unique requestCode, 최소 extras를 함께 검토하십시오.  
  patterns:  
  - pattern: PendingIntent.get$METHOD($CTX, $REQ, $INTENT, $FLAGS)  
  - metavariable-regex:  
    metavariable: $FLAGS  
    regex: ((?i).*FLAG_MUTABLE.*)
```

- 이 규칙은 의도적으로 넓게 FLAG_MUTABLE 사용처를 1차 수집합니다.
- metavariable-regex 의 .*FLAG_MUTABLE.* 는 부분문자열 매칭이라 일부러 느슨하게 두고, 정확도는 AI 분류 단계에서 보강합니다.
- 실제 위험도는 2차 분류에서 봅니다.
- 즉, "mutable이 정말 필요한가", "Intent가 explicit인가", "추가 red flag가 있는가"를 사람이나 AI가 이어서 판단합니다.

예시 1: 검출 결과

```
$ semgrep scan --metrics=off \
--config rules/android-local.yml \
app/src/main/kotlin/com/example/PendingIntentLab.kt
```

1 Code Finding

app/src/main/kotlin/com/example/PendingIntentLab.kt

>>> android-pendingintent-flag-mutable

Mutable PendingIntent입니다.

대부분은 FLAG_IMMUTABLE을 사용해야 하며, mutable이 꼭 필요하다면
explicit component/package, unique requestCode, 최소 extras를 함께 검토하십시오.

```
12 | return PendingIntent.getBroadcast(
```

- 여기서 바로 보는 질문:
 - 이 mutable이 정말 필요한가
 - Intent가 explicit인가
 - requestCode, URI grant, 민감 action이 같이 붙어 있는가

예시 1: 오탐 사례 (합법적인 mutable use case)

```
object ReplyActionFactory {  
  fun buildReply(context: Context, threadId: Int): PendingIntent {  
    val replyIntent = Intent(context, ReplyReceiver::class.java).apply {  
      action = "com.example.ACTION_INLINE_REPLY"  
      setPackage(context.packageName)  
    }  
  
    return PendingIntent.getBroadcast(  
      context,  
      threadId,  
      replyIntent,  
      PendingIntent.FLAG_MUTABLE or PendingIntent.FLAG_UPDATE_CURRENT  
    )  
  }  
}
```

- notification inline reply처럼 시스템이 RemoteInput 결과를 채워야 하는 경우에는 mutable이 실제로 필요할 수 있습니다.
- 이 경우에도 component/package는 고정하고, mutable이 필요한 이유를 PR 설명·코드 코멘트·RFC 중 한 곳에 남겨 둡니다.
- 그래야 다음 분류 사이클에서 동일 패턴을 allowlist로 빠르게 분기할 수 있습니다.

예시 1: AI 분류 포인트

1. 이 use case가 inline reply / bubble / alarm 등으로 mutable이 정말 필요한지 확인한다.
2. Intent가 explicit component 또는 package로 고정돼 있는지 확인한다.
3. requestCode가 고유한지, FLAG_UPDATE_CURRENT가 민감 extra를 덮어쓰지 않는지 본다.
4. URI grant, 민감 action, 내부 receiver/service의 extra 신뢰 여부를 함께 본다.
5. 실제로 필요 없는 mutable이면 true positive로, 합법적 시스템 use case면 allowlist 후보로 기록한다.

예시 1: AI 검토 입력 템플릿

다음 Semgrep finding이 실제 취약점인지, 오탐인지 Android 코드리뷰 관점에서 판별해줘.

```
- check_id: android-pendingintent-flag-mutable
- file: app/src/main/kotlin/com/example/PendingIntentLab.kt:12
- rule_intent: mutable PendingIntent 1차 수집
- code:
  return PendingIntent.getBroadcast(
    context,
    0,
    deleteIntent,
    PendingIntent.FLAG_MUTABLE or PendingIntent.FLAG_UPDATE_CURRENT
  )
```

추가 요청:

1. 실제 취약점 / 오탐 / 추가 확인 필요 중 하나로 분류
2. 그렇게 판단한 근거 3개
3. explicit 여부, requestCode, URI grant, 민감 action 관점에서 더 확인할 call path
4. 규칙 튜닝 포인트 1개와 코드리뷰 코멘트 3줄

- sample 1에서는 `FLAG_MUTABLE` 자체보다 `implicit Intent`, `URI grant`, 민감 action이 함께 붙는지까지 봐야 합니다.
- AI에게는 "실제로 mutable이 필요한 use case인가"를 먼저 묻게 하는 편이 분류 효율이 좋습니다.

예시 2: 무엇을 점검하나

java-android-weak-hash-md5-sha1

- 점검 대상: SHA-1 , MD5 , HmacSHA1 , SHA1withRSA 같은 구식 알고리즘 사용
- 문제 상황: 무결성 검증, 서명 검증, 토큰 서명, 업데이트 검증에 그대로 사용
- 왜 위험한가:
 - 충돌 공격 관점에서 현대 기준에 맞지 않음
 - 보안 결정에 쓰이면 영향이 커짐
- 왜 자주 발견되나: "레거시라서 그대로" 남아 있는 경우가 많아 발견율이 높음

핵심 질문

- 이 해시는 보안 의사결정에 쓰이는가?
- 단순 표시/호환용인가, 검증용인가?
- SHA-256 이상으로 바꿀 수 있는가?

예시 2: 문제 코드

```
package com.example;

import java.security.MessageDigest;
import java.security.Signature;
import java.util.Arrays;

class LegacyCrypto {
    boolean verifyManifest(byte[] manifest, byte[] expectedDigest) throws Exception {
        byte[] actual = MessageDigest.getInstance("SHA-1").digest(manifest);
        return Arrays.equals(actual, expectedDigest);
    }

    Signature newSigner() throws Exception {
        return Signature.getInstance("SHA1withRSA");
    }
}
```

- `verifyManifest()`에서는 SHA-1 결과를 실제 비교 판단에 사용합니다.
- `newSigner()`는 오래된 서명 알고리즘을 그대로 사용합니다.
- 이런 코드는 "레거시 때문에 유지"되는 경우가 많아, 우선 수집이 중요합니다.

예시 2: 비교적 안전한 기본형

```
package com.example;

import java.security.MessageDigest;
import java.security.Signature;

class ManifestVerifier {
    boolean verifyManifest(byte[] manifest, byte[] expectedDigest) throws Exception {
        byte[] actual = MessageDigest.getInstance("SHA-256").digest(manifest);
        return MessageDigest.isEqual(actual, expectedDigest);
    }

    Signature newSigner() throws Exception {
        return Signature.getInstance("SHA256withRSA");
    }
}
```

- 기본 안전선:
 - SHA-256 이상, 서명은 SHA256withRSA / SHA256withECDSA 권장
 - 알고리즘 선택은 정책 한 곳에 모아 두고 호출부에서 분기 금지
- 참고로 비교 함수도 timing-safe 한 `MessageDigest.isEqual` 사용 — 약한 해시와는 별개 이슈지만 같이 다듬어 두면 좋습니다.

예시 2: Semgrep 규칙

```
rules:  
- id: java-android-weak-hash-md5-sha1  
  languages: [java]  
  severity: ERROR  
  message: >  
    MD5·SHA-1 계열의 약한 해시 또는 서명 알고리즘 사용입니다.  
    SHA-256 이상 또는 최신 권장 알고리즘으로 교체하십시오.  
  pattern-either:  
    - pattern: MessageDigest.getInstance("MD5")  
    - pattern: MessageDigest.getInstance("SHA1")  
    - pattern: MessageDigest.getInstance("SHA-1")  
    - pattern: Mac.getInstance("HmacSHA1")  
    - pattern: Signature.getInstance("SHA1withRSA")  
    - pattern: Signature.getInstance("SHA1withDSA")  
    - pattern: Signature.getInstance("SHA1withECDSA")
```

- `pattern-either` : 해시, HMAC, 전자서명 초기화 지점을 넓게 수집합니다.
- 장점: 단순하고 빠르게 찾을 수 있습니다.
- 한계: "보안 검증"에 쓰는지, "표시용"인지는 구분하지 못합니다.
- 그래서 이 룰은 AI 분류 단계와 함께 쓰기 좋습니다.

예시 2: 검출 결과

```
$ semgrep scan --metrics=off \
--config rules/android-local.yml \
app/src/main/java/com/example/LegacyCrypto.java
```

```
| 2 Code Findings |
```

```
app/src/main/java/com/example/LegacyCrypto.java
```

```
>>> java-android-weak-hash-md5-sha1
```

MD5·SHA-1 계열의 약한 해시 또는 서명 알고리즘 사용입니다.

SHA-256 이상 또는 최신 권장 알고리즘으로 교체하십시오.

```
8 | byte[] actual = MessageDigest.getInstance("SHA-1").digest(manifest);
```

```
>>> java-android-weak-hash-md5-sha1
```

```
13 | return Signature.getInstance("SHA1withRSA");
```

- 분석자는 "이 값이 실제 보안 판단에 쓰이는가"를 먼저 봐야 합니다.
- 이 구분을 사람이 직접 하거나 AI에게 맡기면 분류 속도를 높일 수 있습니다.

예시 2: 오탐 사례

```
package com.example;

import java.security.MessageDigest;
import java.security.cert.X509Certificate;

class CertificateScreen {
    String legacyFingerprint(X509Certificate cert) throws Exception {
        byte[] fp = MessageDigest.getInstance("SHA-1").digest(cert.getEncoded());
        return Hex.encode(fp);
    }
}
```

- 이 코드만 보서는 SHA-1 지문이 trust decision에 쓰이는지, 단순 표시·기록용인지 가릴 수 없습니다.
- Semgrep은 호출 자체만 보기 때문에 우선 후보로 잡고, "어디에 쓰이는가"는 다음 단계에서 봅니다.

예시 2: AI 분류 포인트

AI 분류 포인트

1. 해시 결과가 if/allow/deny/verify로 이어지는지 본다.
2. 결과가 UI 표시, 로그, 마이그레이션 보고서로만 가는지 확인한다.
3. 신뢰 판단이 아니라면 낮은 우선순위 또는 호환성 메모로 내린다.
4. 신뢰 판단이라면 실제 취약점으로 분류하고 SHA-256 이상 대체 코드를 제안한다.

예시 2: AI 검토 입력 템플릿

다음 Semgrep finding이 실제 취약점인지, 오타인지 Java 코드리뷰 관점에서 판별해줘.

```
- check_id: java-android-weak-hash-md5-sha1
- file: app/src/main/java/com/example/LegacyCrypto.java:8
- rule_intent: 약한 해시 / 서명 알고리즘 1차 수집
- code:
byte[] actual = MessageDigest.getInstance("SHA-1").digest(manifest);
return Arrays.equals(actual, expectedDigest);
```

추가 요청:

1. 실제 취약점 / 오타 / 추가 확인 필요 중 하나로 분류
2. 그렇게 판단한 근거 3개
3. 이 값이 trust decision, UI 표시, 호환성 경로 중 어디에 쓰이는지 더 확인할 함수 / call path
4. SHA-256 이상 대체 방향과 규칙 튜닝 아이디어 1개

- sample 2에서는 "약한 알고리즘을 썼다"보다 "그 결과가 실제 보안 판단에 쓰이는가"를 먼저 가려내는 것이 중요합니다.
- `check_id` , 호출 코드, 주변 함수, helper 이름까지 함께 주는 것이 중요합니다.

정리

- 룰은 의도적으로 넓게, 정확도는 AI 분류 단계에서 — 이 두 단계 구조가 발표의 핵심입니다.
- 예시 1 (PendingIntent): `FLAG_MUTABLE` 은 1차 수집, AI에게는 "mutable이 정말 필요한 use case인가"를 먼저 묻습니다.
- 예시 2 (weak hash): 약한 해시 호출은 1차 수집, AI에게는 "이 결과가 trust decision에 쓰이는가"를 먼저 묻습니다.
- 두 사례의 공통 패턴 — AI는 코드 자체보다 *사용 목적*과 *call path*를 가리는 데 가장 효과적입니다.
- Semgrep CE + 로컬 룰만으로도 후보 추출은 충분하고, 최종 판정은 사람·AI 분류와 묶을 때 정확도가 올라갑니다.